

Session-Tracked Agentic Software Development: From Spec-Driven Design to Unsupervised Development

A Framework for Traceable, Resumable, and Auditable
LLM-Driven Software Engineering

Terry McGuinness
Environmental Modeling Center
National Oceanic and Atmospheric Administration
Terry.McGuinness@noaa.gov

February 2026

Abstract

Large language model (LLM) coding agents can generate, modify, and analyze source code with increasing proficiency, yet their work remains largely *ephemeral*: each conversation starts from scratch, actions lack structured audit trails, and there is no formal mechanism for an agent to resume interrupted work across sessions or across different client interfaces. We present a **session-tracked agentic development framework** that addresses these limitations through three interlocking contributions: (1) **Spec-Driven Development (SDD)**, a methodology in which machine-readable workflow specifications are the single source of truth for what must be built, validated, and delivered; (2) **Session Management with semantic step tags** (**research**, **design**, **implement**, **configure**, **validate**, **document**, **ingest**), which gives every agent action a durable, classified audit record persisted to disk; and (3) a **two-layer intent architecture** that separates *protocol knowledge* (agent instruction files) from *task knowledge* (per-invocation plans), enabling a progression from interactive supervised development (ISD) to fully unsupervised development (USD). We validate the framework through deployment on the NOAA Global Workflow codebase, where a Phase 27 session completed 8/8 tracked steps—ingesting 9,155 new graph relationships across 383 shell-script nodes—in 5 minutes with a complete, queryable audit trail. The framework is implemented atop the Model Context Protocol (MCP), making it portable across VS Code Copilot, Claude Desktop, Claude Code, and CI/CD pipelines without modification.

Keywords: Agentic AI · Spec-Driven Development · Session Tracking · Unsupervised Development · LLM Software Engineering · Model Context Protocol · Knowledge Graphs · Retrieval-Augmented Generation

Contents

1	Introduction	4
1.1	The Ephemeral Agent Problem	4
1.2	Our Approach: Session-Tracked Agentic Development	4
1.3	Contributions	4
1.4	Paper Organization	5

2	Background and Related Work	5
2.1	LLM Coding Agents	5
2.2	Tool Use and Reasoning	5
2.3	Software Process Frameworks	5
2.4	Knowledge-Enhanced Code Intelligence	5
2.5	Human-AI Interaction	6
3	Spec-Driven Development	6
3.1	Core Principles	6
3.2	Workflow Specification Format	6
3.3	Phase Naming Convention	7
4	Session Management Model	7
4.1	Design Goals	7
4.2	State Architecture	7
4.3	Session Lifecycle	7
4.4	Semantic Step Tags	8
4.5	History Event Schema	8
4.6	Cross-Conversation Resume	9
5	Two-Layer Intent Architecture	9
5.1	Protocol Layer: Instruction Files	10
5.2	Task Layer: Plans and Specifications	10
5.3	Intent Propagation	10
6	Execution Mode Spectrum	11
6.1	Step Classification	11
6.2	From ISD to USD	11
7	Implementation Architecture	12
7.1	Technology Stack	12
7.2	SessionManager Implementation	12
7.3	MCP Tool Registration	12
7.4	Multi-Client Portability	13
7.5	Docker MCP Gateway	13
8	Empirical Evaluation	13
8.1	Case Study: Phase 27F-G Shell Graph Ingestion	13
8.1.1	Session Metadata	13
8.1.2	Step-by-Step Trace	13
8.1.3	Tag Distribution	14
8.1.4	Quantitative Results	14
8.1.5	Audit Trail Verification	15
8.2	Session Management Validation	15
8.3	Comparison with Untracked Development	15

9 Discussion	15
9.1 Limitations	15
9.1.1 Single Active Session	15
9.1.2 Completion Snapshot Loss	15
9.1.3 No Step Reordering	16
9.1.4 Tag Vocabulary Evolution	16
9.2 Generalizability	16
9.3 Comparison with Existing Approaches	16
9.4 Future Work	17
10 Conclusion	17
A Complete SessionManager.js API Reference	20
B History Event Types	20
C Semantic Tag Vocabulary	20
D Phase 27F-G Complete History Trace	21

1 Introduction

1.1 The Ephemeral Agent Problem

LLM-based coding agents—GitHub Copilot, Claude Code, Cursor, Aider, SWE-agent [20]—have demonstrated remarkable capability on benchmarks such as SWE-bench [11]. Yet in practice, deploying these agents on production codebases exposes a fundamental gap: *agent work is ephemeral*.

1. **No persistent state.** Each conversation is a blank slate. If the user closes VS Code, all context is lost; the agent cannot resume.
2. **No structured audit trail.** Actions are buried in chat transcripts. There is no machine-queryable record of *what* was done, *why*, or *in what order*.
3. **No cross-client portability.** Work started in VS Code cannot be continued in a terminal agent or a CI/CD pipeline without manual context transfer.
4. **No intent classification.** An agent generating code and an agent researching architecture are indistinguishable in the conversation record.

These limitations are acceptable for ad hoc question-answering but untenable for *systematic software development*—multi-step engineering tasks that span hours or days, require traceable decisions, and must survive interruptions.

1.2 Our Approach: Session-Tracked Agentic Development

We propose a framework that wraps LLM agent interactions in a **session management layer** inspired by database transaction semantics and workflow orchestration patterns [18]. The core abstraction is:

Session

A **session** is a bounded unit of development work that tracks a named *phase*, records individual *steps* with semantic tags, persists state to disk, and produces a durable *history log* that survives session completion.

Sessions are managed by four MCP [3] tools: `start_sdd_session`, `record_sdd_step`, `get_sdd_session` (resume), and `complete_sdd_session`. These tools are exposed identically to every MCP-compatible client, achieving cross-client portability by construction.

1.3 Contributions

1. **Spec-Driven Development (SDD)** methodology with machine-readable Markdown workflow specifications and phased execution (Section 3).
2. **Phase 31 Session Model:** a lightweight, file-based session manager with seven semantic step tags and append-only history (Section 4).
3. **Two-layer intent architecture** separating protocol instructions from task plans, enabling the progression from supervised to unsupervised execution (Section 5).
4. **Execution mode spectrum:** `dry_run` → `supervised` → `auto_approved` → `batch`, providing progressive trust levels (Section 6).
5. **Empirical validation** on the NOAA Global Workflow codebase with a complete 8-step session trace (Section 8).
6. **Multi-client MCP architecture** portable across VS Code, Claude Desktop, terminal agents, and CI/CD (Section 7).

1.4 Paper Organization

Section 2 surveys related work. Section 3 presents the SDD methodology. Section 4 details the session management model. Section 5 describes the two-layer intent architecture. Section 6 defines the execution mode spectrum. Section 7 covers the MCP-based implementation. Section 8 presents empirical results. Section 9 discusses limitations and future work. Section 10 concludes.

2 Background and Related Work

2.1 LLM Coding Agents

The rapid advancement of code-capable LLMs [7, 14] has spawned a generation of *agentic* tools that go beyond autocompletion to execute multi-step development tasks. SWE-agent [20] introduced an agent-computer interface (ACI) design that achieves state-of-the-art performance on SWE-bench. MetaGPT [9] and ChatDev [13] explore multi-agent architectures where specialized roles (architect, developer, tester) collaborate on software projects.

These systems demonstrate that LLMs can plan and execute complex engineering tasks, but they share a common limitation: **session state is in-memory and ephemeral**. When the process terminates, all execution context is lost.

2.2 Tool Use and Reasoning

ReAct [21] established the paradigm of interleaving reasoning traces with tool invocations. Toolformer [15] showed that LLMs can learn to call external tools autonomously. Chain-of-thought prompting [19] demonstrated that explicit reasoning steps improve task performance.

Our framework builds on these foundations but adds a critical missing element: **tool calls that record their own execution in a persistent, structured audit log**. In our model, the tool `record_sdd_step` is simultaneously an *action* (the agent records progress) and an *observation* (the updated session state is returned for context).

2.3 Software Process Frameworks

Agile methodologies [6], test-driven development (TDD) [5], and requirements engineering [16] provide structured processes for human developers. Continuous delivery [10] automates the build-test-deploy pipeline.

SDD adapts these concepts for LLM agents: specifications replace user stories, session state replaces sprint boards, and semantic step tags replace story point categories. The fundamental insight is that **LLM agents need process discipline just as human teams do**—not to constrain their generative capability, but to make their work traceable, resumable, and auditable.

2.4 Knowledge-Enhanced Code Intelligence

Graph-based code analysis [1] has a rich history. GraphRAG [8] extends retrieval-augmented generation [12] with graph-derived community summaries. Our earlier work demonstrated hybrid Neo4j+ChromaDB retrieval for operational weather forecasting code, achieving 3.8× improvement in Precision@10 over text search baselines.

The session management framework presented here is *orthogonal* to RAG architecture: it tracks *how* the agent uses these tools, not *what* the tools return. However, the combination is powerful: RAG provides context, and session tracking records how that context was applied.

2.5 Human-AI Interaction

Guidelines for human–AI interaction [2] emphasize that AI systems should support user control over system actions and make clear what the system can do. Studies of LLM coding tools [17] reveal that developers value transparency in AI decision-making.

Our execution mode spectrum (Section 6) directly implements these principles: **supervised** mode provides per-step approval gates, while **batch** mode delegates full autonomy only when trust has been established through prior supervised sessions.

3 Spec-Driven Development

SDD Principle

“If it’s not in the SDD, it doesn’t get coded.” Every development task begins with a machine-readable specification that defines objectives, steps, deliverables, and acceptance criteria before any code is written.

3.1 Core Principles

SDD is built on four foundational principles:

- P1: Specifications as First-Class Artifacts.** Development tasks are expressed as structured workflows in Markdown before implementation begins. Specifications define *what* (objectives), *how* (validation criteria), *when* (phase ordering), and *who* (role assignment).
- P2: Progressive Refinement.** Specifications evolve from high-level intent to detailed implementation through iterative passes, each adding precision while maintaining consistency with the parent specification.
- P3: Health-Integrated Architecture.** All system components integrate health monitoring from initialization. The agent can verify preconditions (database connectivity, file system access) before attempting any step.
- P4: Supervised Human–AI Collaboration.** AI agents propose implementations based on specifications; human experts review, refine, and approve. The level of supervision is configurable along the execution mode spectrum.

3.2 Workflow Specification Format

SDD workflows are expressed as Markdown documents with structured metadata. We chose Markdown over YAML or JSON for three reasons: (1) human readability, (2) compatibility with version control diffs, and (3) native rendering in documentation platforms.

A workflow specification contains:

```

1 # Phase Name
2 **Version**: X.Y  **Author**: Name  **Date**: YYYY-MM-DD
3
4 ## Objective
5 High-level goal and scope definition.
6
7 ## Steps
8 ### Step 1: Step Name
9 **Type**: research | design | implement | configure | validate
10 **Deliverables**: Concrete outputs
11 **Acceptance Criteria**: Pass/fail conditions

```

```

12
13 ### Step 2: ...
14     (repeat for each step)
15
16 ## Success Metrics
17 Quantifiable completion criteria.

```

Listing 1: SDD Workflow Specification Structure

3.3 Phase Naming Convention

Phases follow the pattern `phase<N><letter>_<descriptor>.md`, enabling hierarchical decomposition:

- `phase27_jjob_script_rag_enhancement.md` — parent phase
- `phase27a` through `phase27g` — sub-phases A–G
- `phase27h_multi_collection_routing.md` — spin-off issue

The framework currently contains 31+ phases with sub-phases, spanning infrastructure, tooling, knowledge ingestion, and methodology.

4 Session Management Model

4.1 Design Goals

The session manager was designed with five goals:

1. **Lightweight:** No database required; file-based persistence.
2. **Cross-conversation:** Survives server restarts and client reconnections.
3. **Cross-client:** Works identically from VS Code, Claude Desktop, or CLI.
4. **Auditable:** Append-only history log for compliance and reconstruction.
5. **Simple API:** Four tools covering the full lifecycle.

4.2 State Architecture

Session state is maintained in two complementary files:

active_session.json Contains the current session state: session ID, phase name, start time, current step, completed steps array, skipped steps, and notes. Written on every `record_sdd_step` call. Deleted on `complete_sdd_session`.

history.jsonl Append-only event log in JSON Lines format. Every lifecycle event—`started`, `step_completed`, `step_skipped`, `resumed`, `completed`, `abandoned`—is appended as an independent line. Never truncated or overwritten.

This dual-file architecture mirrors the write-ahead log (WAL) pattern in database systems: `active_session.json` is the “current state” and `history.jsonl` is the “redo log.” Even if the active session file is lost, the complete session can be reconstructed from the history.

4.3 Session Lifecycle

Figure 1 illustrates the session lifecycle as a state machine.



Figure 1: Session lifecycle state machine. The `get_sdd_session` tool reads but does not modify state, enabling resume across conversations.

4.4 Semantic Step Tags

Each recorded step carries a **semantic tag** drawn from a fixed vocabulary of seven values:

Tags serve three purposes:

1. **Intent classification.** The tag tells *what kind* of work the step represents, independent of its specific content.
2. **Workflow analysis.** Aggregating tags across sessions reveals development patterns (e.g., ratio of `implement` to `validate` steps).
3. **Automation gating.** In future work, the execution mode could auto-approve `research` and `validate` steps while requiring human approval for `implement` steps.

4.5 History Event Schema

Each event appended to `history.jsonl` has the structure:

```

1 {
2   "sessionId": "session_2026-02-19_8aioyi",
3   "phase": "phase27_jjob_script_rag_enhancement",
4   "event": "step_completed",
5   "step": 5,
6   "name": "Run shell graph ingestion",
7   "tag": "implement",
8   "notes": "235 scripts processed, 383 ShellScript nodes,
9           9155 new relationships",

```

Table 1: Semantic step tags with intent classification and examples.

Tag	Intent Class	Example
research	Discovery	Read existing code, explore architecture
design	Planning	Draft data model, choose algorithm
implement	Construction	Write code, modify configuration
configure	Setup	Set environment variables, install dependencies
validate	Verification	Run tests, check Neo4j counts, dry-run
document	Knowledge	Update README, write changelog entry
ingest	Enrichment	Run graph ingestion, update ChromaDB

```

10  "timestamp": "2026-02-19T20:49:30.641Z"
11  }

```

Listing 2: History event for a step completion

The schema is deliberately flat—no nesting, no foreign keys—to maximize grep-ability and ensure that every line is independently parseable. This design choice favors operational simplicity over relational normalization.

4.6 Cross-Conversation Resume

The `get_sdd_session` tool reads `active_session.json` from disk and returns the full session state, including all completed steps. Because the file persists across server restarts, an agent in a *new* conversation can resume precisely where the previous conversation left off:

Algorithm 1 Cross-conversation resume protocol

- 1: Agent starts in new conversation
 - 2: Agent calls `get_sdd_session()`
 - 3: **if** active session exists **then**
 - 4: Read completed steps $\{s_1, \dots, s_k\}$
 - 5: Read total steps N from specification
 - 6: Continue from step $k + 1$
 - 7: **else**
 - 8: No active session; start fresh or report idle
 - 9: **end if**
-

This mechanism requires **no conversation memory**, **no external database**, and **no user intervention**. The agent discovers its own state through a tool call.

5 Two-Layer Intent Architecture

A critical challenge in agentic development is communicating *both* the development methodology (“how we build software”) and the specific task (“what to build right now”) to the LLM. We address

this with a **two-layer intent stack**:

Table 2: Two-layer intent architecture.

Layer	Scope	Mechanism	Content
Protocol	Persistent	Instruction files	SDD methodology, tool usage, session lifecycle, conventions
Task	Per-invocation	/plan or spec	Phase objectives, step details, acceptance criteria

5.1 Protocol Layer: Instruction Files

Instruction files (e.g., `.github/instructions/*.md`) are loaded into the LLM’s system context on every invocation. They encode:

- **SDD methodology**: “If it’s not in the SDD, it doesn’t get coded.”
- **Session protocol**: Start sessions before work; record steps with semantic tags; complete sessions with summaries.
- **Tool usage**: Parameter names, required vs. optional arguments, common workflows.
- **Conventions**: ASCII-only console output, 2-space indentation, SPOT (Single Point of Truth) for configuration.

The LLM internalizes these instructions as *behavioral constraints* that apply to every task, regardless of specifics. This is analogous to a developer internalizing a team’s coding standards.

5.2 Task Layer: Plans and Specifications

The task layer provides per-invocation context through two mechanisms:

Explicit plans (/plan): The user provides a natural-language plan that the LLM decomposes into an internal task list. The plan is ephemeral—it exists only in the current conversation context.

Workflow specifications: A Markdown specification (Section 3) provides structured step definitions. The agent reads the spec via `get_sdd_workflow` and auto-generates its own internal plan from the structured content.

Key Insight

The protocol layer teaches the agent *how to work*; the task layer teaches the agent *what to build*. Together, they provide sufficient intent for the agent to self-plan, self-tag, self-validate, and self-resume without human prompting at each step.

5.3 Intent Propagation

The complete intent stack for any given agent action is:

$$\text{Intent}(a) = \underbrace{\text{Spec}(p)}_{\text{What to build}} \oplus \underbrace{\text{Tag}(s)}_{\text{What kind of work}} \oplus \underbrace{\text{Name}(s) + \text{Notes}(s)}_{\text{Specifics}} \quad (1)$$

where p is the phase specification, s is the current step, and $\oplus$ denotes context concatenation. Each component adds a layer of semantic precision:

- **Spec:** “Phase 27: J-Job Script RAG Enhancement”
- **Tag:** `implement` (this is a construction step)
- **Name:** “Run shell graph ingestion”
- **Notes:** “235 scripts processed, 383 ShellScript nodes, 9,155 new relationships”

This layered structure means that even if the name and notes are terse, the combination of spec + tag + name provides enough context to reconstruct the full intent post hoc.

6 Execution Mode Spectrum

The framework defines four execution modes with **progressive trust levels**:

Table 3: Execution mode spectrum. Trust increases left to right.

Mode	Side Effects	Human	Audit	Use Case
<code>dry_run</code>	None	None	Preview only	Planning, validation
<code>supervised</code>	Yes	Per-step	Full	Active development
<code>auto_approved</code>	Yes	Manifest	Full	Trusted workflows
<code>batch</code>	Yes	None	Full	CI/CD pipelines

6.1 Step Classification

Steps are classified by their side-effect potential:

Read-only (`research`, `validate`): Auto-execute in all modes. These steps query databases, read files, and check conditions without modifying system state.

Side-effect (`implement`, `configure`, `ingest`): Require approval in `supervised` mode, pre-approved by manifest in `auto_approved` mode, or fully autonomous in `batch` mode.

Documentation (`document`, `design`): Typically auto-approved since they produce artifacts (Mark-down, diagrams) that are easily reviewed and reverted.

6.2 From ISD to USD

The execution mode spectrum enables a natural progression from **Interactive Supervised Development (ISD)** to **Unsupervised Development (USD)**:

1. **ISD Phase:** The developer runs sessions in `supervised` mode, reviewing each side-effect step. The session history accumulates a record of what actions were approved and which patterns are trustworthy.
2. **Trust Calibration:** After multiple successful supervised sessions, the developer identifies step types and workflow patterns that consistently produce correct results.
3. **USD Phase:** Trusted patterns are codified in an auto-approval manifest. The agent executes autonomously within these boundaries while still recording every step to the audit log.

Key Insight

USD is not “unsupervised” in the sense of unmonitored. Every step is still recorded with full semantic tagging and notes. The difference is *when* the human reviews: in ISD, review is synchronous (before each step); in USD, review is asynchronous (after completion, via the audit log).

7 Implementation Architecture

7.1 Technology Stack

The framework is implemented as part of a unified MCP server:

Table 4: Technology stack.

Component	Technology	Purpose
MCP Server	Node.js (ES Modules)	Tool orchestration via MCP SDK
Session Manager	<code>SessionManager.js</code>	Session lifecycle and history
Graph Database	Neo4j 5.x	Code structure relationships
Vector Database	ChromaDB 0.4+	Semantic search embeddings
Transport	stdio / Streamable HTTP	MCP client communication
State Storage	JSON + JSONL files	Session persistence

7.2 SessionManager Implementation

The `SessionManager` class (260 LOC) implements the full session lifecycle with seven public methods:

```

1 class SessionManager {
2   startSession(phaseName, options) // -> session state
3   recordStep(stepNumber, name, tag, notes) // -> updated state
4   skipStep(stepNumber, reason) // -> updated state
5   getSessionState() // -> state | null
6   resumeSession() // -> state (with timestamp)
7   completeSession(summary) // -> final state
8   abandonSession(reason) // -> final state
9 }

```

Listing 3: SessionManager API

Key implementation details:

- **Single active session constraint.** `startSession` throws if an active session already exists. This prevents interleaved sessions which would complicate the audit trail.
- **Duplicate detection.** `recordStep` rejects duplicate step numbers, preventing accidental re-recording.
- **Tag validation.** Unknown tags produce a warning but do not block recording, allowing graceful evolution of the tag vocabulary.
- **Auto-parsing step count.** `startSession` reads the workflow specification file and counts `### Step N:` patterns to auto-populate `totalSteps`.
- **Duration calculation.** `completeSession` computes wall-clock duration from start to completion timestamps.

7.3 MCP Tool Registration

The four session tools are registered with the MCP server alongside 38+ other tools (code analysis, semantic search, compliance checking):

Table 5: SDD session tools: MCP registration.

Tool	Required Params	Description
<code>start_sdd_session</code>	<code>phase</code>	Start tracked session; optional <code>totalSteps</code> , <code>notes</code>
<code>record_sdd_step</code>	<code>step</code> , <code>name</code>	Record completion; optional <code>tag</code> , <code>notes</code>
<code>get_sdd_session</code>	(none)	Return active session state for resume
<code>complete_sdd_session</code>	(none)	Complete session; optional <code>summary</code>

7.4 Multi-Client Portability

Because session state is persisted to disk (not in MCP server memory), the framework is inherently multi-client:

1. A developer starts a session in **VS Code Copilot** (stdio transport) and completes steps 1–3.
2. The developer closes VS Code.
3. Later, a **Claude Code** terminal agent calls `get_sdd_session()`, discovers steps 1–3 complete, and continues from step 4.
4. A **CI/CD pipeline** calls `complete_sdd_session()` after automated validation.

This is possible because all clients connect to the same MCP server instance (or containers reading the same volume mount), and the session state is a simple JSON file on a shared filesystem.

7.5 Docker MCP Gateway

For production deployment, the MCP server runs inside a Docker container behind the Docker MCP Gateway (Streamable HTTP on port 18888). The `sdd_framework/` directory is volume-mounted, ensuring session state survives container restarts:

```

1 volumes:
2   - /path/to/sdd_framework:/app/sdd_framework # state persists
3   - /path/to/supported_repos:/app/supported_repos # read-only

```

Listing 4: Volume mount for session persistence

8 Empirical Evaluation

8.1 Case Study: Phase 27F-G Shell Graph Ingestion

We evaluate the framework through a complete session trace from Phase 27 of the NOAA Global Workflow RAG enhancement project. This phase involved ingesting shell script structure (J-Jobs, ex-scripts, environment variables) into the Neo4j graph database.

8.1.1 Session Metadata

8.1.2 Step-by-Step Trace

Table 7 shows the complete step trace as recorded in `history.jsonl`.

Table 6: Phase 27F-G session metadata.

Attribute	Value
Session ID	session_2026-02-19_8aioyi
Phase	phase27_jjob_script_rag_enhancement
Total Steps	8 (auto-parsed from specification)
Completed Steps	8/8 (100%)
Duration	5 minutes
Client	VS Code Copilot (stdio)

Table 7: Complete step trace for Phase 27F-G.

#	Tag	Name	Key Metrics
1	implement	Fix shell graph password	SPOT-compliant default
2	implement	Add CLI flags	-dry-run, -clear, -verbose
3	implement	Fix bridge paths	Duplicate session, path refactor
4	validate	Dry-run shell graph	235 scripts, 0 errors
5	implement	Run ingestion	383 nodes, 9,155 relationships
6	implement	Re-run bridges	8 cross-language edges
7	validate	Validate Neo4j counts	40,413 total nodes (+206)
8	validate	Validate MCP tools	5/6 tests pass, 1 miss

8.1.3 Tag Distribution

The session’s tag distribution reveals the development pattern:

- **implement**: 5 steps (62.5%)—majority of effort in code changes
- **validate**: 3 steps (37.5%)—significant validation effort
- **research**: 0 steps—the phase spec provided sufficient context

This ratio (roughly 2:1 implement-to-validate) is characteristic of a well-specified phase where the agent does not need to “explore”—the specification told it exactly what to build, and the validation steps confirmed correctness.

8.1.4 Quantitative Results

Table 8: Knowledge graph metrics before and after Phase 27F-G.

Metric	Before	After	Delta
Total nodes	40,207	40,413	+206
ShellScript nodes	0	383	+383
ShellFunction nodes	0	63	+63
EnvironmentVariable nodes	n/a	2,489	+2,489
SOURCES relationships	148	393	+245
INVOKES relationships	4	352	+348
DEPENDS_ON_ENV	6,007	7,225	+1,218
DEFINES relationships	9,690	9,753	+63
Total new relationships	—	—	+9,155

8.1.5 Audit Trail Verification

The complete session is reconstructable from `history.jsonl` alone. We verified this by:

1. Parsing all events with `sessionId = session_2026-02-19_8aioyi`
2. Confirming `started` event with correct phase name
3. Confirming 8 `step_completed` events in order
4. Confirming `completed` event with duration and step counts
5. Total: 10 events (1 started + 8 step_completed + 1 completed)

8.2 Session Management Validation

Prior to the Phase 27 session, two validation sessions tested the session lifecycle:

1. **Phase 31 review test** (`session_2026-02-18_8p8fsr`): 2 steps completed, 1 step skipped, verifying the skip and completion mechanisms.
2. **Health check test** (`session_2026-02-19_x8kyst`): 2 steps completed with an explicit `resumed` event, verifying the cross-conversation resume capability.

All three sessions are permanently recorded in `history.jsonl` (17 total events).

8.3 Comparison with Untracked Development

Without session tracking, the same Phase 27 work would have produced:

- A chat transcript (ephemeral, not machine-queryable)
- Git commits (capture *what* changed, not *why* or *in what order*)
- No structured record of the dry-run step that validated correctness before the live ingestion
- No tag-based analysis of implementation vs. validation effort

The session trace adds approximately 3 KB of structured metadata (10 JSONL events) that makes the entire development process reconstructable and analyzable.

9 Discussion

9.1 Limitations

9.1.1 Single Active Session

The current design enforces one active session at a time. This prevents interleaved work (e.g., pausing Phase 27 to fix an urgent bug in a different phase). The workaround is to **abandon** the current session and start a new one, but this loses the in-progress state.

Proposed solution: Named session stacks, where sessions can be suspended and resumed by name, similar to Git stash.

9.1.2 Completion Snapshot Loss

When `complete_sdd_session` is called, the active session file is deleted. While all events persist in `history.jsonl`, the assembled session object (with its complete step array) is lost. To reconstruct the full session state, one must filter and reassemble events from the history file.

Proposed solution: Write a snapshot entry to `history.jsonl` containing the full session state before deletion.

9.1.3 No Step Reordering

Steps must be recorded in increasing numerical order (enforced by duplicate detection). Real-world development sometimes requires revisiting earlier steps. The current design handles this by recording a new step with a higher number and explanatory notes.

9.1.4 Tag Vocabulary Evolution

The fixed tag vocabulary (`research`, `design`, `implement`, `configure`, `validate`, `document`, `ingest`) may not capture all development activities. Unknown tags produce warnings but are accepted, providing a graceful evolution path.

9.2 Generalizability

While developed for NOAA Global Workflow, the framework is domain-agnostic. Any codebase that can be described by Markdown specifications can use SDD session tracking. The MCP transport layer means any LLM with MCP support can participate.

The semantic step tags are also generalizable: `research`, `design`, `implement`, `validate`, and `document` apply to virtually any software project. The `configure` and `ingest` tags are more specific to infrastructure and knowledge-base maintenance but could be replaced with domain-appropriate tags.

9.3 Comparison with Existing Approaches

Table 9: Comparison with existing development frameworks.

Feature	<i>SDD (this work)</i>	<i>SWE-agent</i>	<i>MetaGPT</i>	<i>Claude Code</i>	<i>GitHub Actions</i>
Persistent sessions	✓				
Semantic step tags	✓				
Cross-client resume	✓				
Append-only audit log	✓				✓
Progressive trust modes	✓			✓	
Spec-first methodology	✓		✓		✓
Multi-agent support			✓		
Benchmark evaluation		✓	✓		

SWE-agent [20] and MetaGPT [9] excel at benchmark tasks but lack persistent session management. Claude Code [4] provides progressive trust (auto-accept patterns) but does not persist state across invocations. GitHub Actions provides audit trails and spec-driven workflows but operates at a different abstraction level (CI/CD pipelines rather than interactive development).

Our framework uniquely combines persistent sessions, semantic tagging, cross-client portability, and progressive trust in a single lightweight architecture.

9.4 Future Work

Multi-session support: Named session stacks for concurrent work across multiple phases.

Completion snapshots: Write full session state to history on completion, eliminating reconstruction overhead.

Tag-based auto-approval: In `auto_approved` mode, automatically approve steps tagged `research` or `validate` while gating `implement` steps.

Session analytics dashboard: Aggregate tag distributions, duration patterns, and completion rates across all historical sessions.

Multi-agent coordination: Multiple agents working on the same phase from different clients, with conflict detection based on step-number reservation.

Stale session detection: Automatically flag sessions with no activity beyond a configurable TTL, enabling cleanup of abandoned sessions.

LLM self-evaluation: Use the session audit trail as fine-tuning data: train models to predict which tag and notes format produces the most useful history.

10 Conclusion

We have presented a session-tracked agentic development framework that transforms ephemeral LLM coding interactions into structured, resumable, and auditable engineering processes. The framework’s three core contributions—Spec-Driven Development methodology, the Phase 31 session management model with semantic step tags, and the two-layer intent architecture—address fundamental gaps in current LLM agent tooling.

Empirical evaluation on the NOAA Global Workflow codebase demonstrates the framework’s practical value: a Phase 27 session completed 8/8 tracked steps in 5 minutes, ingesting 9,155 new knowledge graph relationships across 383 shell-script nodes, with a complete machine-queryable audit trail of 10 events in the append-only history log.

The execution mode spectrum (`dry_run` → `supervised` → `auto_approved` → `batch`) provides a principled path from interactive supervised development to unsupervised autonomy, with trust calibrated through experience rather than configuration.

Perhaps most importantly, the entire framework is implemented as four MCP tools totaling 260 lines of code, persisting state in two files on a shared filesystem. This simplicity is a feature, not a limitation: it means the framework can be adopted incrementally, runs on any platform with a filesystem, and requires no external services beyond what the LLM agent already uses.

Key insight: The gap between current LLM agent capability and production-grade software engineering is not primarily a capability gap—it is a *process* gap. LLMs can write code, analyze architectures, and validate results. What they lack is a structured process for recording *what they did*, *why they did it*, and *where they left off*. Session-tracked agentic development fills this gap.

Acknowledgments

This work was developed at the NOAA Environmental Modeling Center as part of the Global Workflow modernization effort. The authors thank the operational forecasting staff for domain expertise and the Parallel Works team for HPC infrastructure support.

References

- [1] Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. A survey of machine learning for big code and naturalness. *ACM Computing Surveys*, 51(4):1–37, 2018.
- [2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, et al. Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019.
- [3] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2024. Open protocol for AI-tool integration.
- [4] Anthropic. Claude code: Agentic coding in the terminal. <https://docs.anthropic.com/en/docs/claude-code>, 2025.
- [5] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2003.
- [6] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, et al. Manifesto for agile software development. 2001. <https://agilemanifesto.org>.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. Codex / GitHub Copilot foundation.
- [8] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. Microsoft Research.
- [9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuhao Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Zhong Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.
- [10] Jez Humble and David Farley. Continuous delivery: Reliable software releases through build, test, and deployment automation. 2010.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [13] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. ChatDev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2024.

- [14] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2024.
- [15] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] Ian Sommerville. Software engineering. 2011.
- [17] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. 2022.
- [18] Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, Bartek Kiepuszewski, and Alistair P. Barros. Workflow patterns. *Distributed and Parallel Databases*, 14(1):5–51, 2003.
- [19] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [20] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.

A Complete SessionManager.js API Reference

Table 10: SessionManager method signatures and semantics.

Method	Parameters	Side Effects
startSession	phaseName: string options: {totalSteps, notes}	Writes active_session.json Appends started to history
recordStep	stepNumber: int name: string tag: string notes: string	Updates active_session.json Appends step_completed to history
skipStep	stepNumber: int reason: string	Updates active_session.json Appends step_skipped to history
getSessionState	(none)	Read-only; returns session or null
resumeSession	(none)	Updates lastActivityAt Appends resumed to history
completeSession	summary: string	Appends completed to history Deletes active_session.json
abandonSession	reason: string	Appends abandoned to history Deletes active_session.json

B History Event Types

Table 11: All event types recorded in history.jsonl.

Event	Additional Fields
started	phase
step_completed	step, name, tag, notes
step_skipped	step, reason
resumed	(none beyond base fields)
completed	summary, completedSteps, skippedSteps, totalSteps, duration
abandoned	reason, completedSteps

All events share base fields: sessionId, phase, event, timestamp.

C Semantic Tag Vocabulary

The tag vocabulary is defined as a constant in SessionManager.js:

```

1 const VALID_TAGS = [
2   'research',    // Discovery: read code, explore architecture
3   'design',      // Planning: draft models, choose algorithms

```

```

4      'implement',    // Construction: write code, modify config
5      'configure',   // Setup: environment, dependencies, infra
6      'validate',    // Verification: tests, counts, dry-runs
7      'document',    // Knowledge: README, changelog, comments
8      'ingest',      // Enrichment: graph ingestion, vector updates
9  ];

```

Tags are validated with a warning (not an error) for unknown values, allowing the vocabulary to evolve without breaking existing sessions.

D Phase 27F-G Complete History Trace

The following is the verbatim content of `history.jsonl` entries for session `session_2026-02-19_8aioyi`, demonstrating the complete audit trail:

Event 1 (session start):

```

1  {"sessionId":"session_2026-02-19_8aioyi",
2   "phase":"phase27_jjob_script_rag_enhancement",
3   "event":"started",
4   "timestamp":"2026-02-19T20:46:46.502Z"}

```

Event 2 (step 1 — implement):

```

1  {"sessionId":"session_2026-02-19_8aioyi",
2   "phase":"phase27_jjob_script_rag_enhancement",
3   "event":"step_completed", "step":1,
4   "name":"Fix shell graph password default",
5   "tag":"implement",
6   "notes":"Changed NEO4J_PASSWORD default from 'password'
7   to 'gfsworkflow2025'. SPOT compliance.",
8   "timestamp":"2026-02-19T20:47:15.678Z"}

```

Event 6 (step 5 — implement, key ingestion):

```

1  {"sessionId":"session_2026-02-19_8aioyi",
2   "phase":"phase27_jjob_script_rag_enhancement",
3   "event":"step_completed", "step":5,
4   "name":"Run shell graph ingestion",
5   "tag":"implement",
6   "notes":"235 scripts processed (0 errors), 383 ShellScript
7   nodes (89 J-Jobs, 130 ex-scripts), 2489 EnvironmentVariables,
8   63 ShellFunctions, 1 ConfigFile. Relationships: 393 SOURCES,
9   352 INVOKES, 1184 EXPORTS, 7225 DEPENDS_ON_ENV,
10   1 READS_CONFIG = 9,155 total new relationships.",
11  "timestamp":"2026-02-19T20:49:30.641Z"}

```

Event 10 (session completion):

```

1  {"sessionId":"session_2026-02-19_8aioyi",
2   "phase":"phase27_jjob_script_rag_enhancement",
3   "event":"completed",
4   "completedSteps":8, "skippedSteps":0, "totalSteps":8,
5   "duration":"5m",
6   "timestamp":"2026-02-19T20:52:02.928Z"}

```